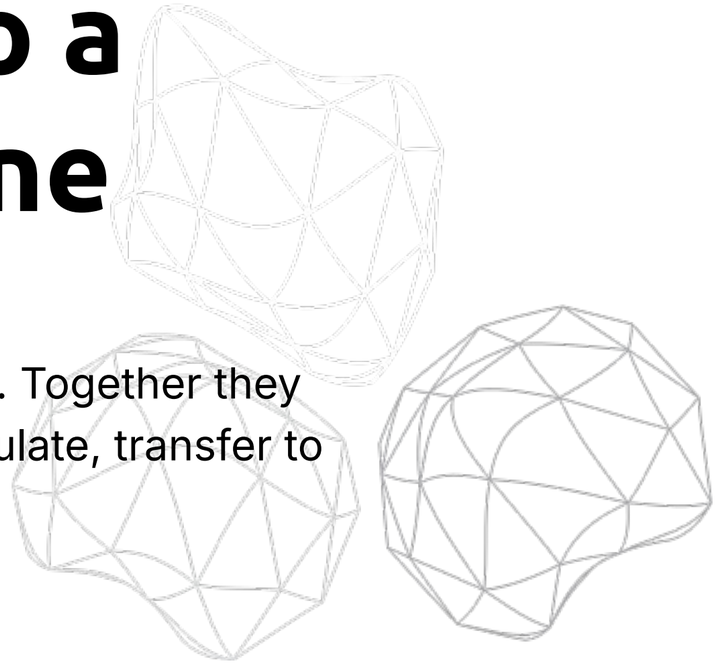
Navigating Machines with Computer Vision

From pixels to a racing drone: how a machine learns to see, practises in simulation and flies in the real world.

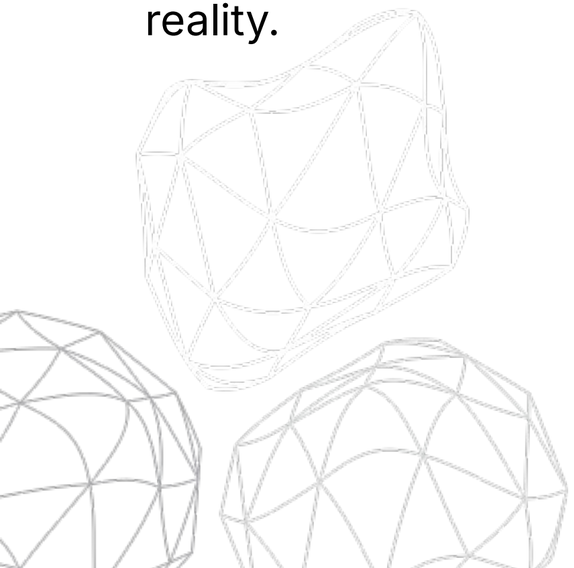
Andrii Yablonskyi, Mykhailo Dvali, Hlib Tataryntsev, Andrii Tkhorenko

Team presentation

Four steps to a flying machine



Each block answers one question. Together they make one pipeline: see, train, simulate, transfer to reality.



1 Seeing

How a computer turns pixels into objects: classification, detection, segmentation.

2 Training

How the model learns: supervised learning, reinforcement learning, fine-tuning.

3 Simulating

Why a drone learns in a virtual world first, and how to make that world realistic.

4 Into reality

Why a model perfect in simulation crashes in reality, and how to fix it.

What Computer Vision actually is?

Machine can see image, But what to do with that images?

What it can see?

What we need from it?

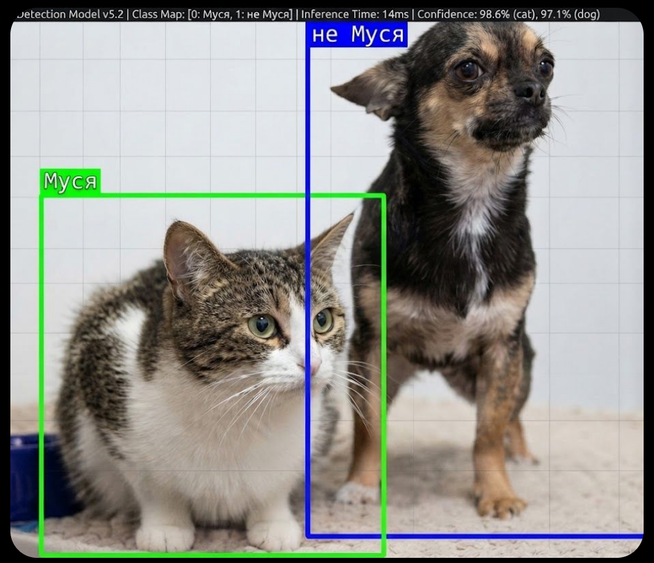
How to make it usefull?

Main tasks in CV



Classification

What object is in the image?



Detection

Where are objects located?

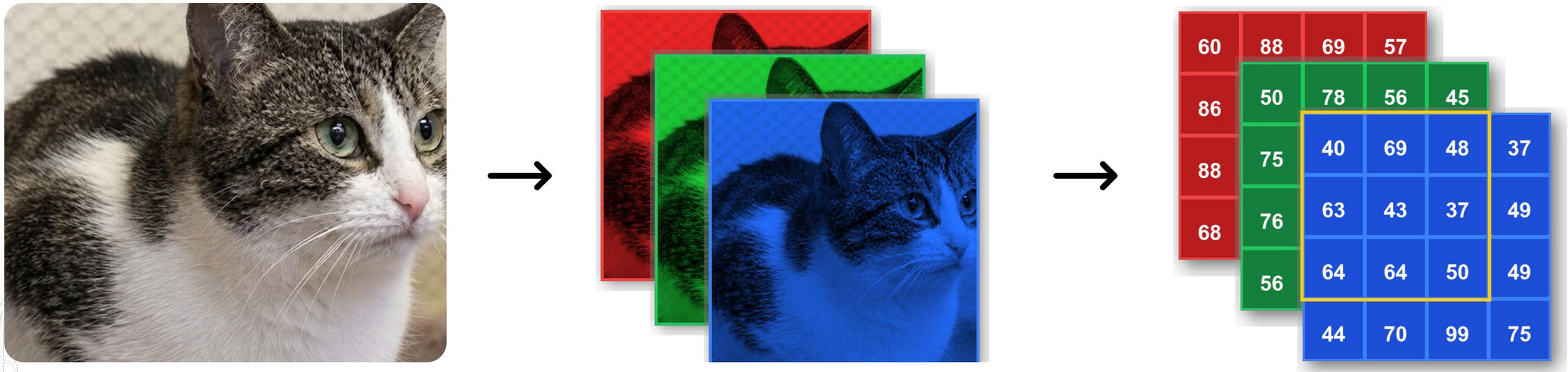


Segmentation

Which pixels belong to what?

Усі вони схожі, але такі різні

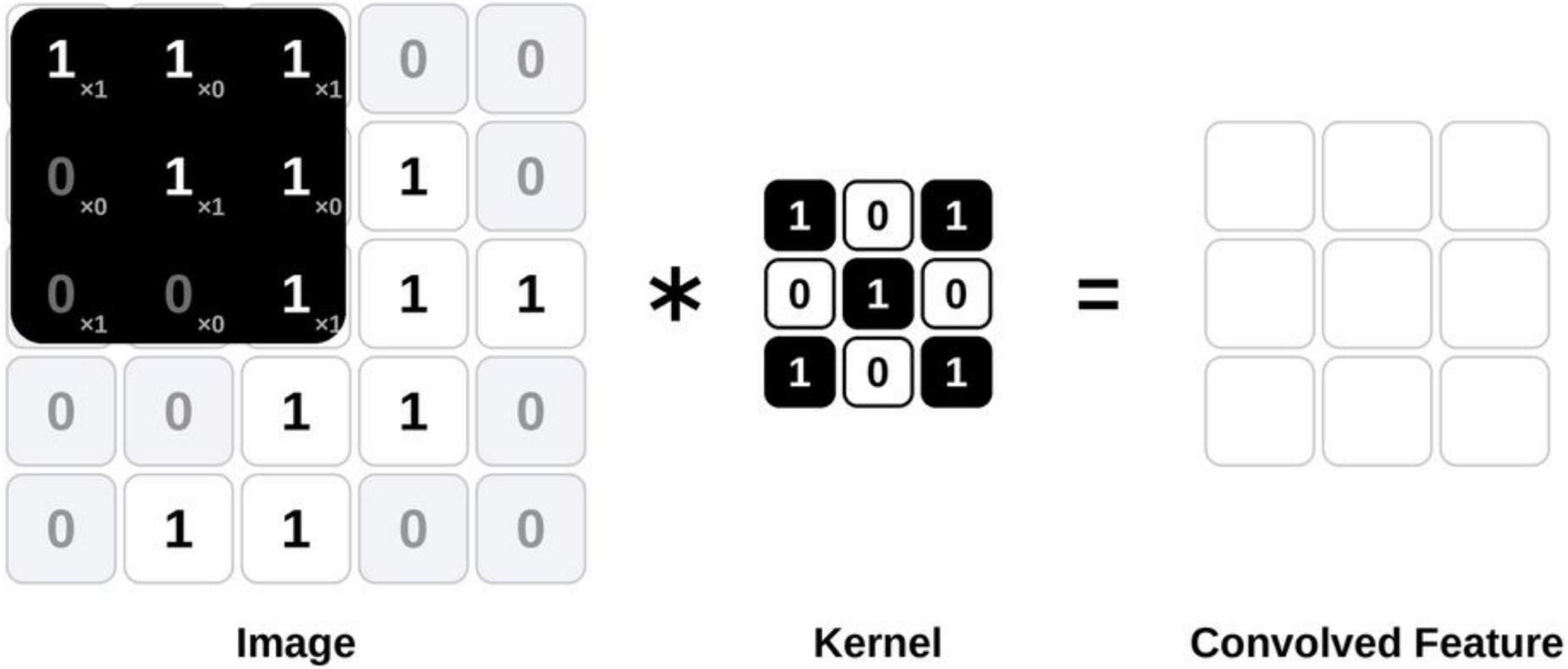
How machines “see” images



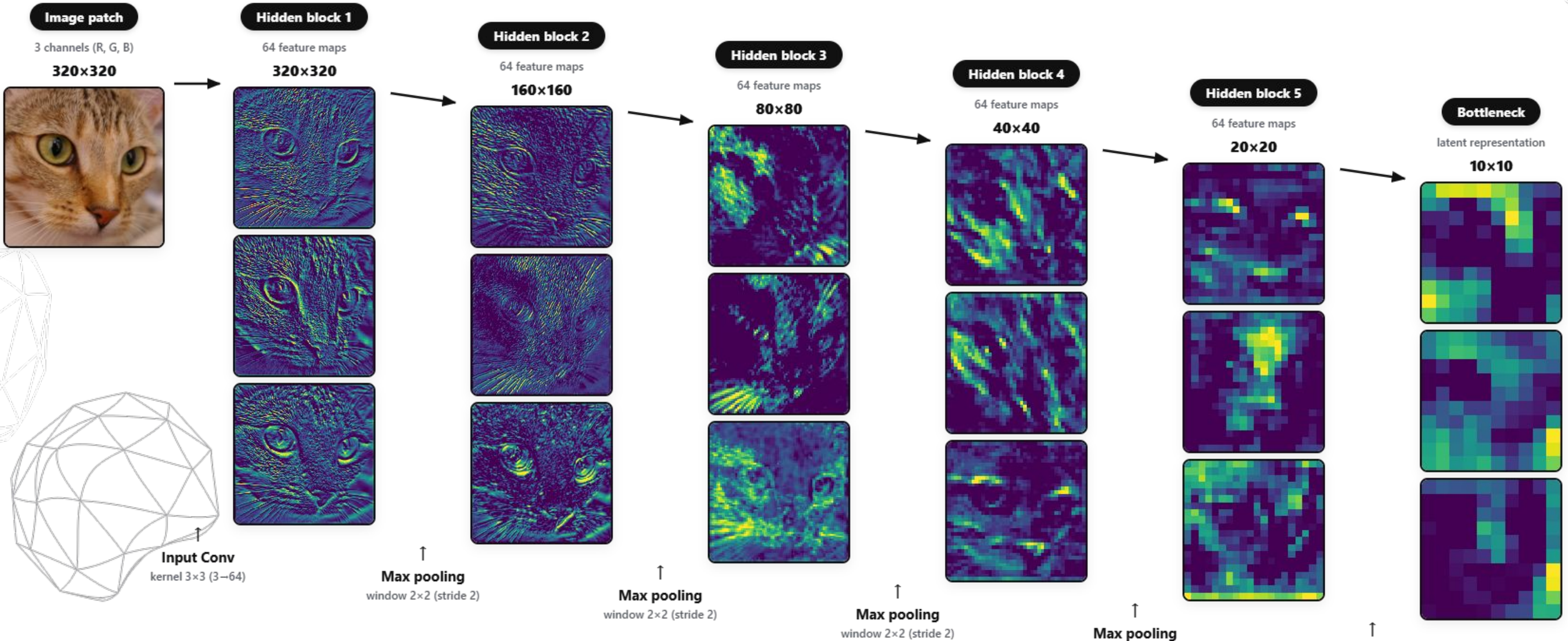
Human perception → **RGB Channels** → **Numeric Matrices**

Before any convolutional filter can slide or detect patterns, an image is transformed into an array of numbers (0-255) stacked across **Red, Green, and Blue** channels.

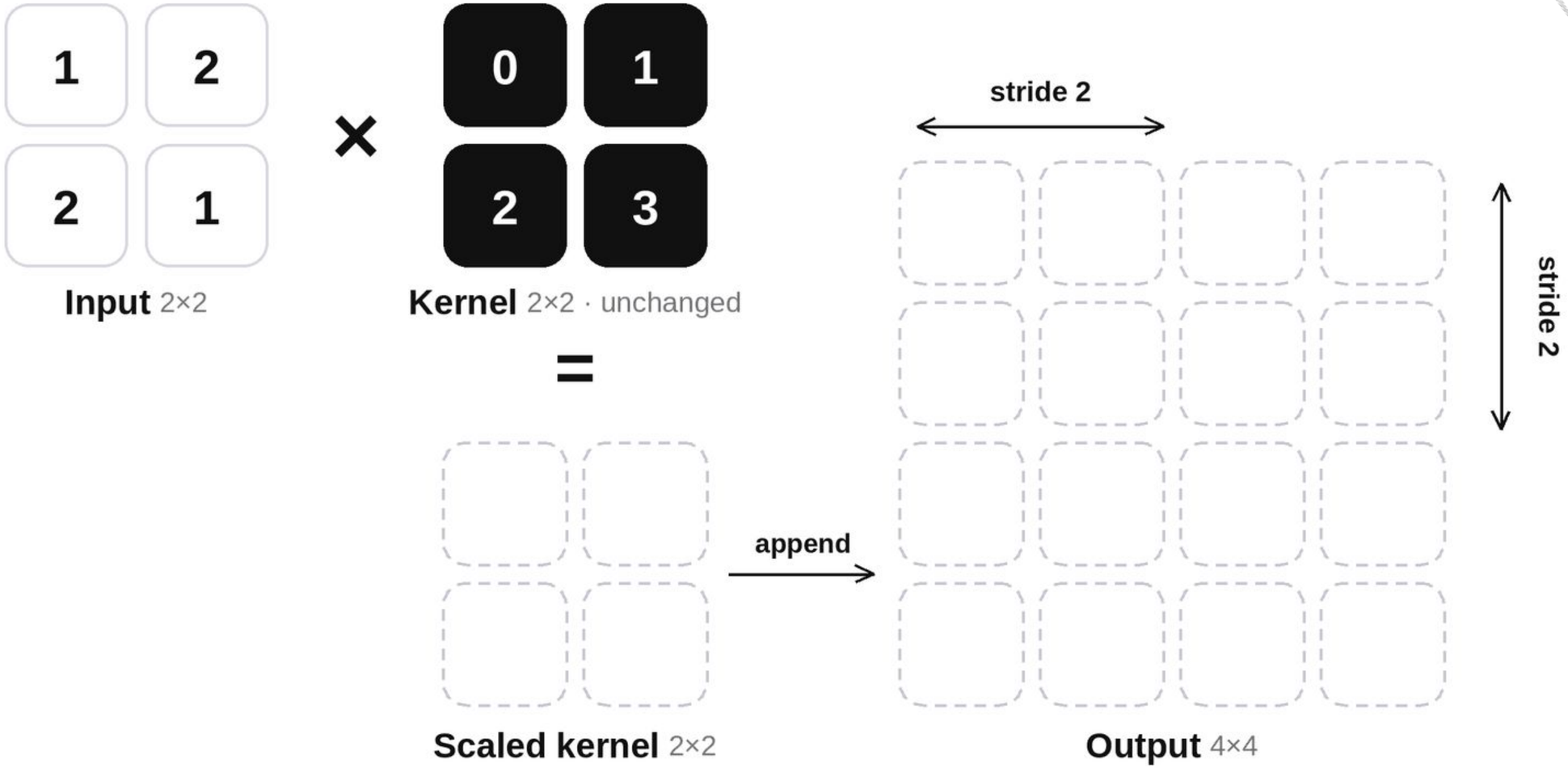
Convolution Explained



Convolution Explained

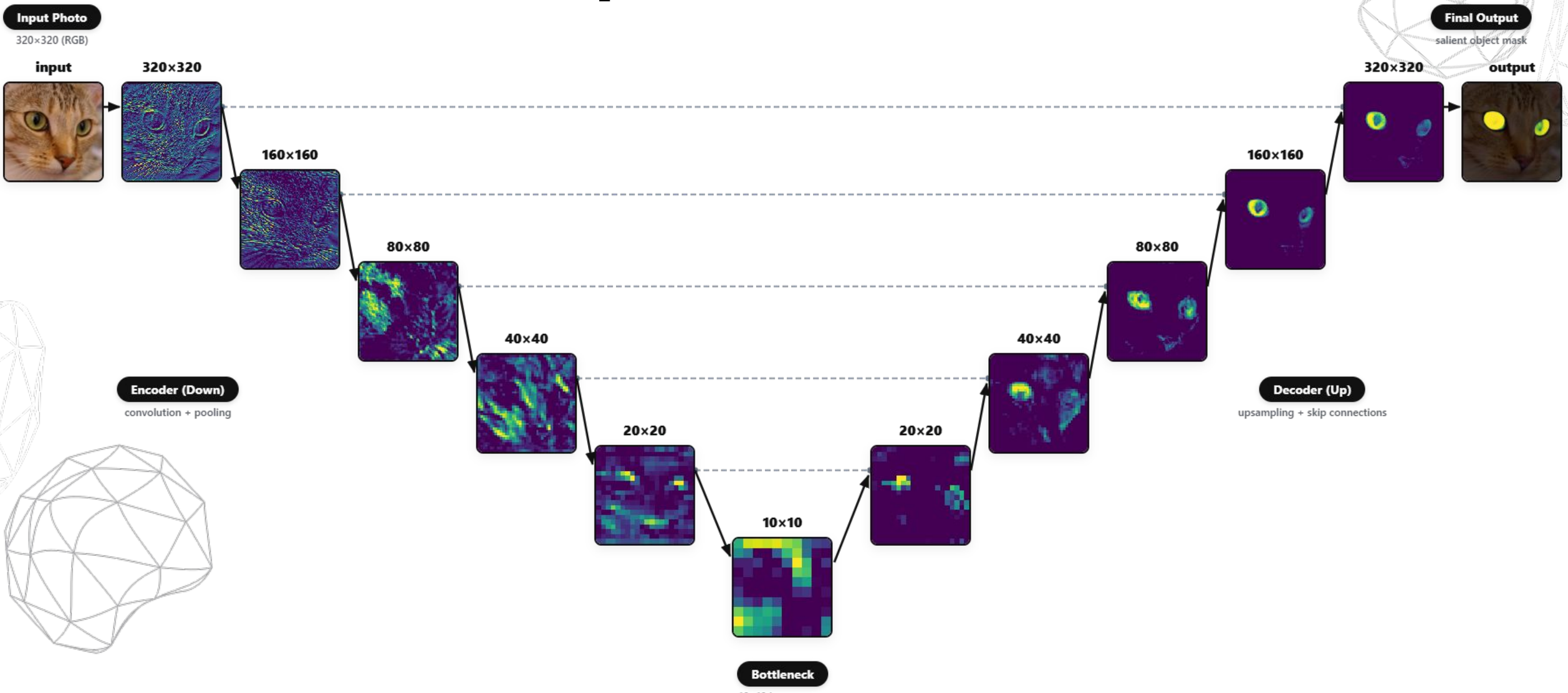


Transposed Convolution



Step 1: multiply a pixel by the kernel · Step 2: append the result to the output

Transposed Convolution



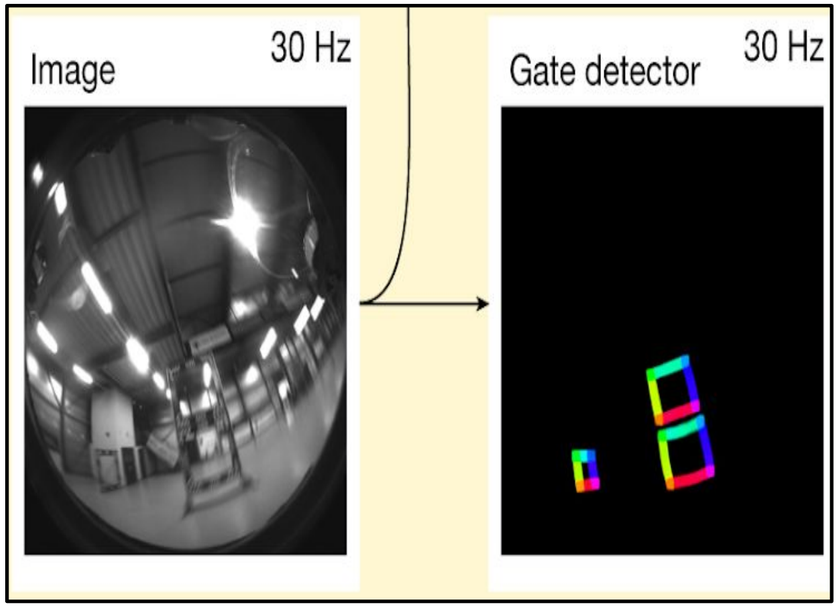


Photo → guess → compare with label → nudge

Labeled images: from the simulator (next part).

Supervised learning

Loss: cross-entropy

One number for how wrong a guess is. A confident wrong guess costs a lot.

Optimization: gradient descent

Nudge the weights (the network's adjustable numbers) so the mistake shrinks.

Swift's U-Net (answers for every pixel) gives each pixel a corner probability.

Sources: Nature, 2023 (Kaufmann et al., Fig. 2, CC BY 4.0); AlphaPilot, arXiv 2005.12813

Reinforcement learning

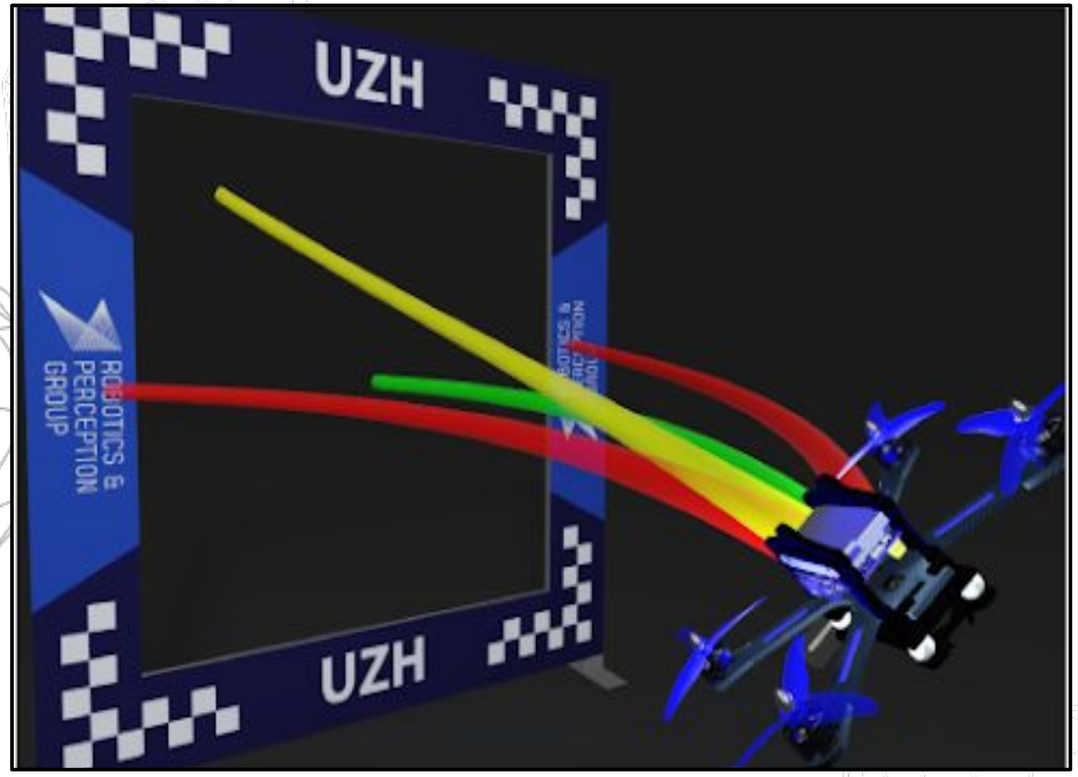
Reward

A score after each step: progress is good, crashes are bad.

Policy gradient

The policy is the network that picks actions. Make well-scored actions likelier.

PPO (Proximal Policy Optimization) keeps every update small, so one bad step cannot ruin the policy.



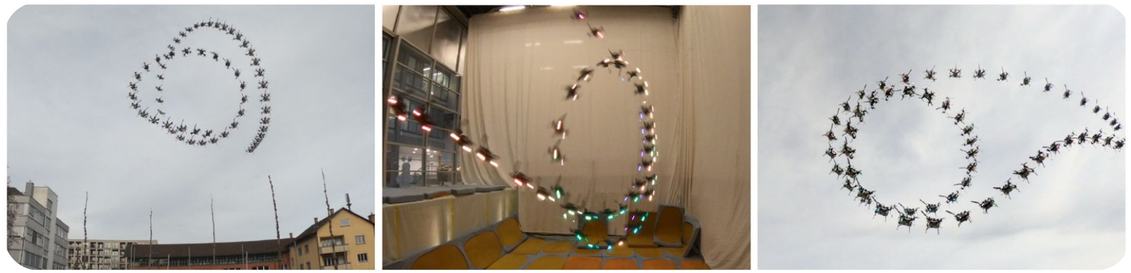
Fly → score → do more of what worked

Simulated flights, scored by the reward

Sources: Nature, 2023 (Kaufmann et al., Fig. 2b, CC BY 4.0); PPO, arXiv 1707.06347

Imitation learning

Learn by copying an expert. No score (reinforcement) and no labelled photo (supervised): the example is what the expert did.



Barrel roll, power loop, Matty flip: trained only in simulation

Teacher: the expert. It sees the exact state (simulation only)

↓

Student: sees only the camera and copies the teacher's actions

↓

DAgger: the student flies, the teacher corrects where it drifted

Training data = what the expert does in each situation

Invent or copy?

Should a machine invent its own strategy, or copy a human?

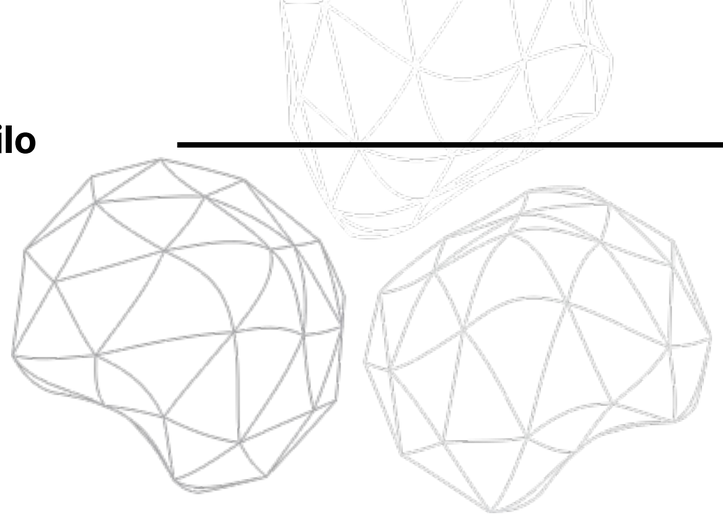
Sources: Nature, 2023 (Kaufmann et al.); Deep Drone Acrobatics, RSS 2020 (arXiv 2006.05768); DAgger (arXiv 1011.0686)

Learning in Swift



Imitation learning is not part of Swift. The same lab used it for acrobatic flips, with a teacher that sees the exact state in simulation.

Sources: Nature, 2023 (Kaufmann et al.); Deep Drone Acrobatics, RSS 2020 (arXiv 2006.05768)



Simulations

Why

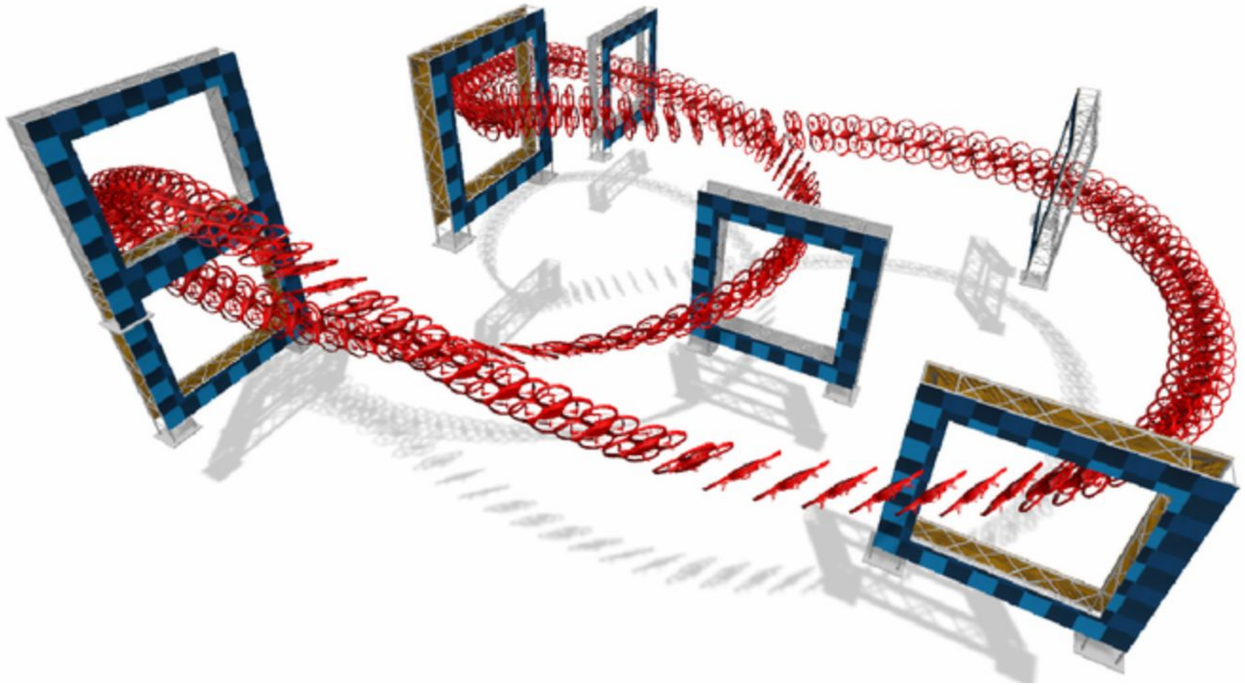
To train a model you need data. You can either find it on the internet, create it yourself in physical world or generate it in a digital one

Then why simulation

Internet - doesn’t always have the data you need and the data isn’t flexible

Generating in physical world - expensive. You need hardware, develop it, replace components after crashes

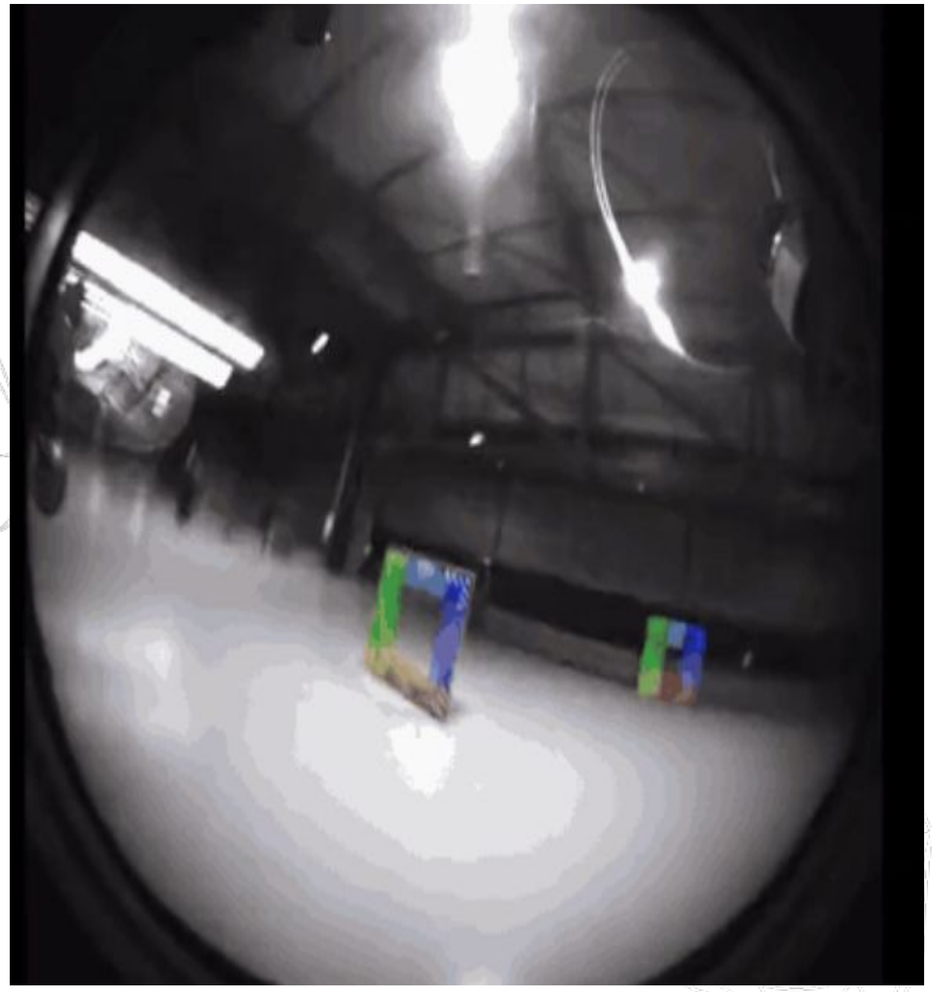
Simulation isn’t expensive - you just need a server and a bit of software. It is also very customizable for your needs



Basic simulation

Take some 3D physics engine, put a drone and a small environment. Here you are - a very basic simulator

Accounting for unaccounted

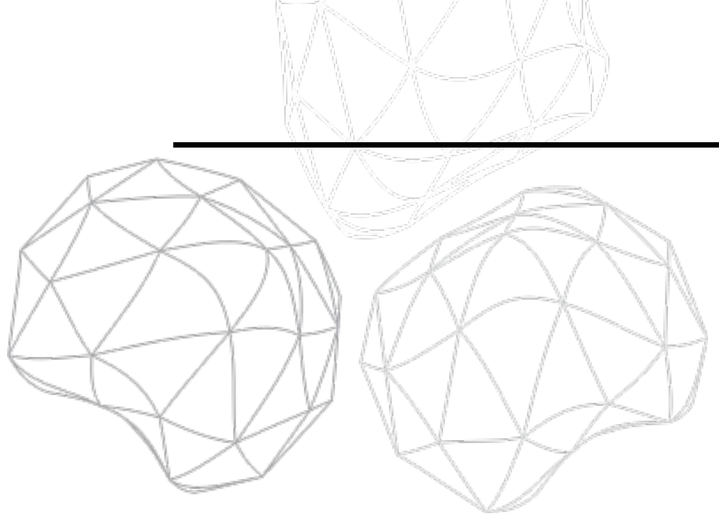


Basic noise

Grain (matrcies noise,basically on every camera)
Lens distortion: Straight lines get curved (you can notice this on many of the fpv drone footages)
Lens flare/dirty lens (відблиски)

Drone specific noise

Motion blur
Rolling shatter (sometimes, it basically is tied to the way the images are processed on some cameras that make it distored in specific way when moving)
Shake
Latency

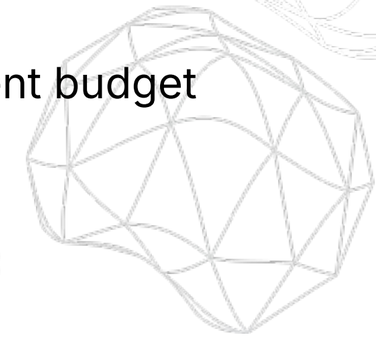


Digital Environment

By hand

- Takes a lot of human hours (creating 3d objects is hard)
- Expensive
- Can create any scene given sufficient budget





If we somehow stole from the real world

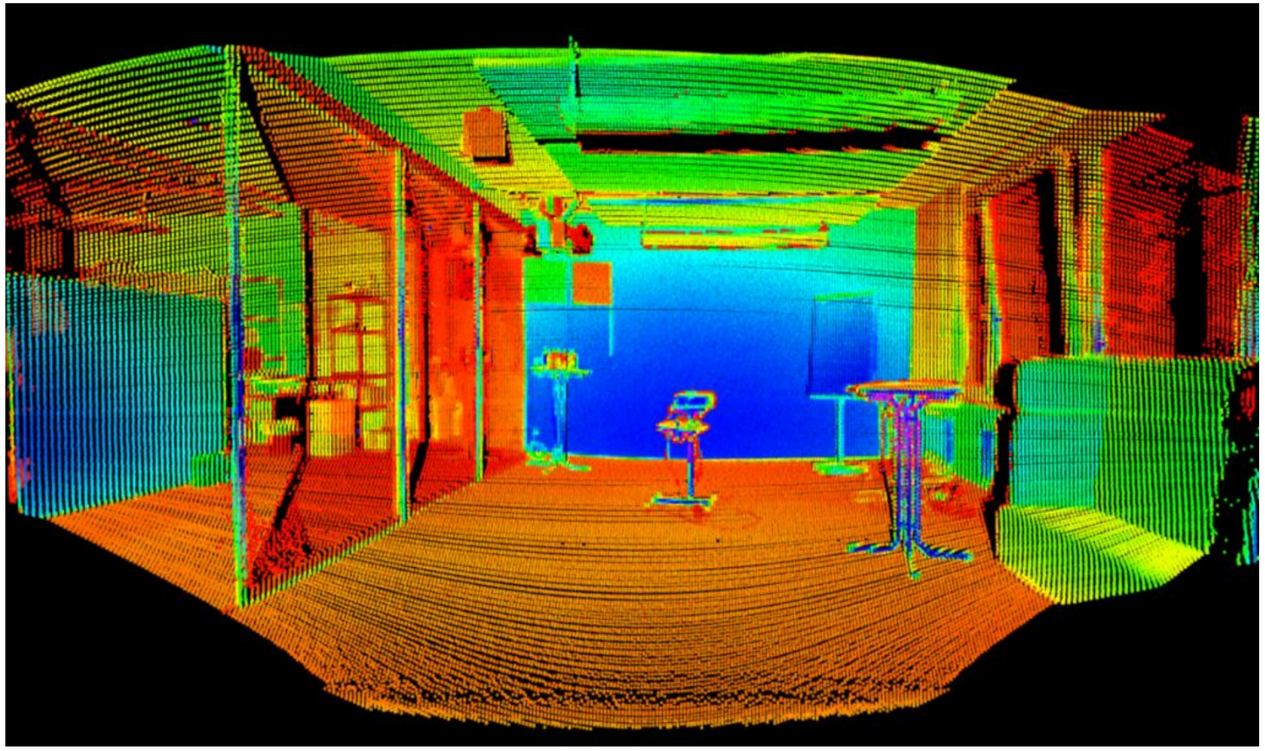
- Less human hours
- Cheaper
- If you can get the environment you are gonna work in it will be more accurate

Option 1
Reconstruct from photos/videos



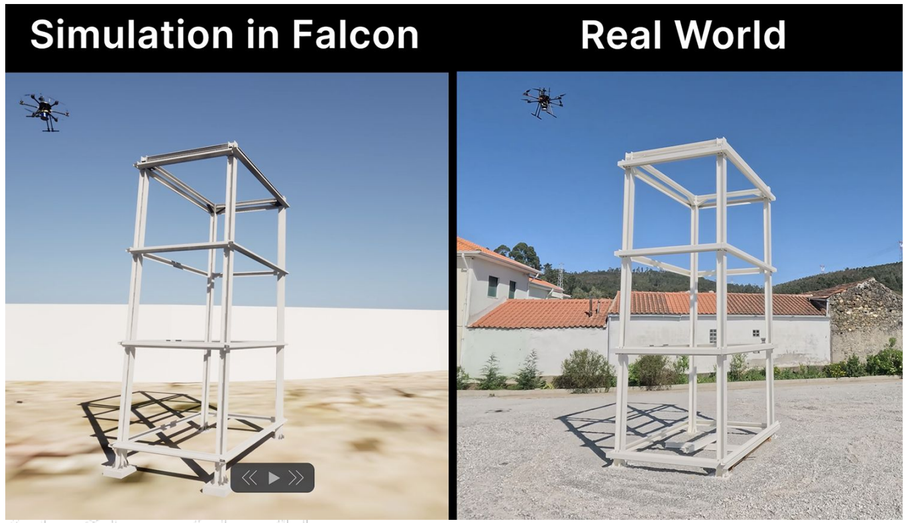


Option 2
Reconstruct from Lidar





The sim-to-real gap



1 · In simulation

80-95% flights succeed on three tasks.

2 · In reality

Only 1 in 10 flights reach the goal

3 · Why

Dirt, motion blur, light, textures, camera noise, small physics errors

Same drone policy, measured in GRaD-Nav (2025). At 20+ m/s, small differences add up to a crash.

Two ways across the gap



Randomize the world

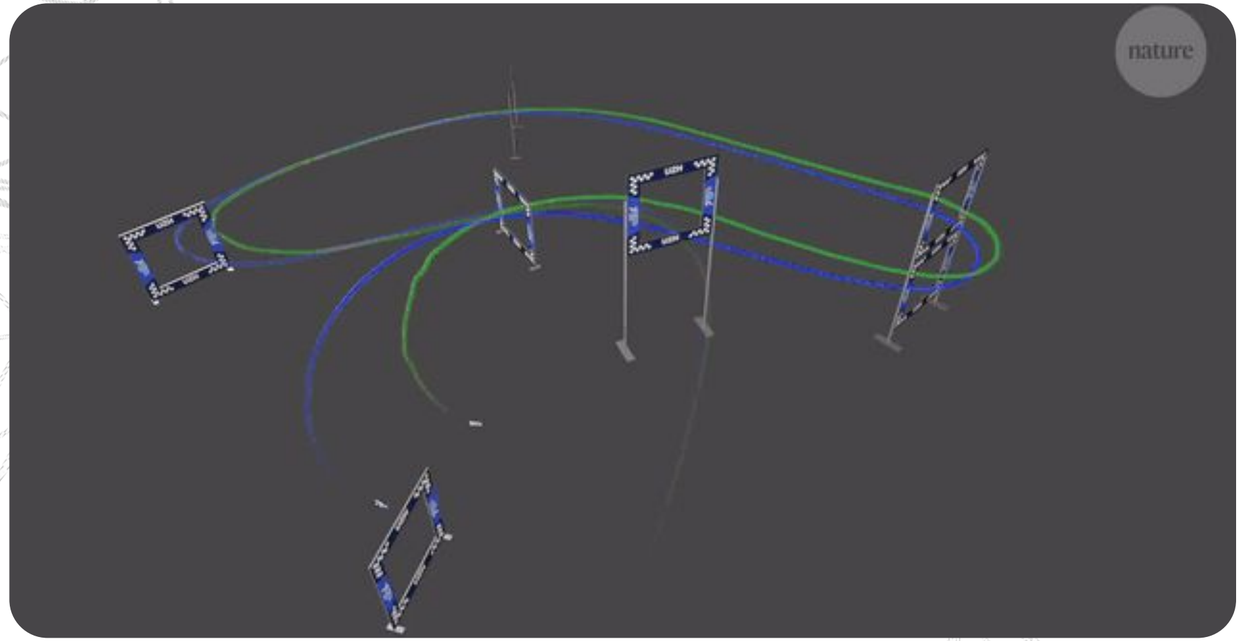
Thousands of varied worlds: odd colours, light, physics. Reality is one more.

Simplify the picture

Convert real images into one clean simulator style (image translation, RCAN).

Both work, but the hard part moves to the module that cleans up reality.

Swift vs MonoRace



Two champion-level racing drones: opposite recipes, same result.

Both

See numbers, not pixels: position, speed, next gate. Camera + motion sensor fused.

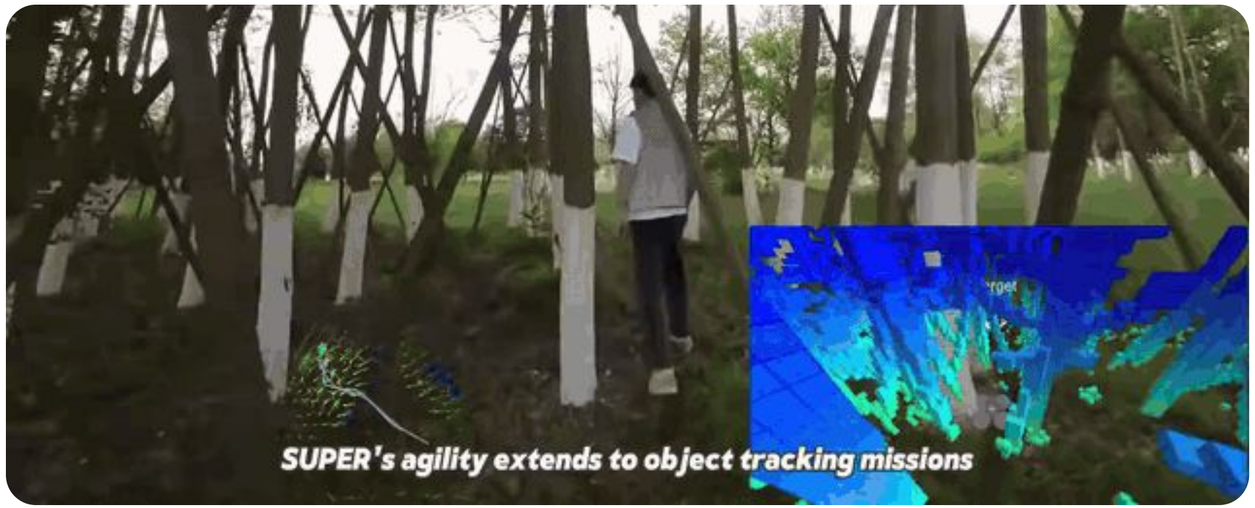
Swift · UZH 2023

VIO + 50 s of real flight to correct the simulator. Won 15 of 25 races.

MonoRace · TU Delft 2025

No real flight data, physics randomized by 40%+. Beat 3 champion-level pilots.





Cameras or LiDAR?

Cameras still struggle with glare, darkness and other drones: 4 of Swift's 10 lost races were collisions. SUPER uses LiDAR and flies over 20 m/s, day and night.

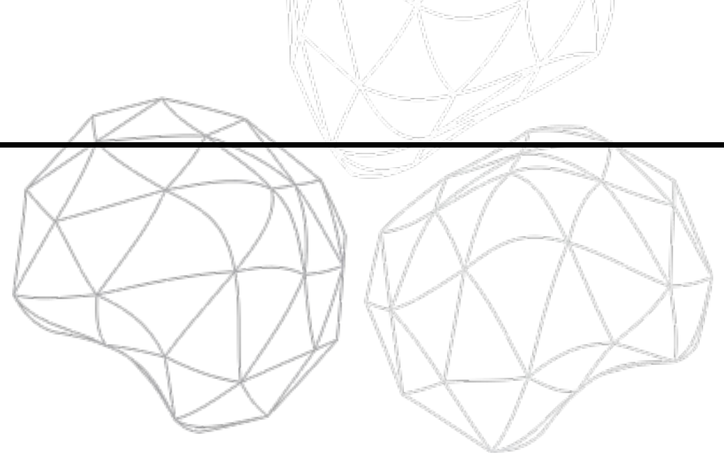
From pixels to the real world

Seeing, training, simulating and adapting: together they give us a machine that can see and navigate in the real world.

1 · See

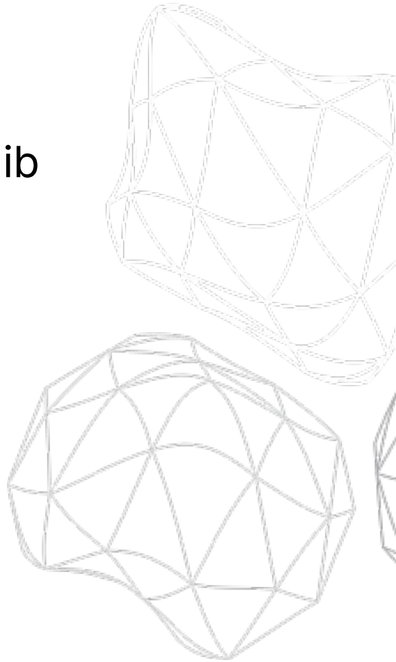
2 · Train · 3 · Simulate

4 · Adapt to reality



Thank You!
Questions?

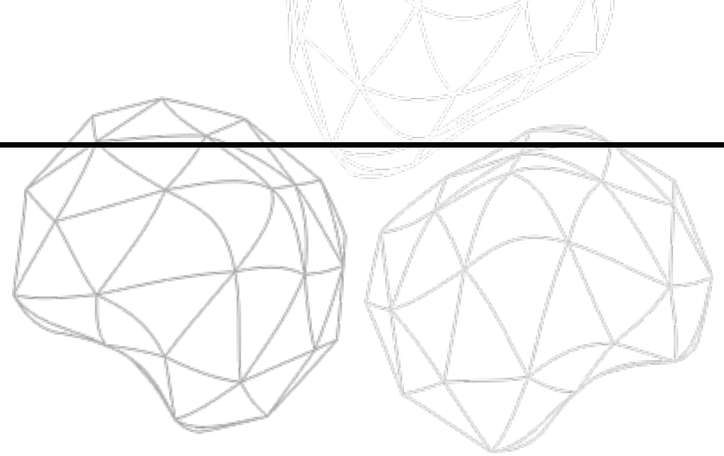
Andrii Yablonskyi · Mykhailo Dvali · Hlib Tataryntsev · Andrii Tkhorenko



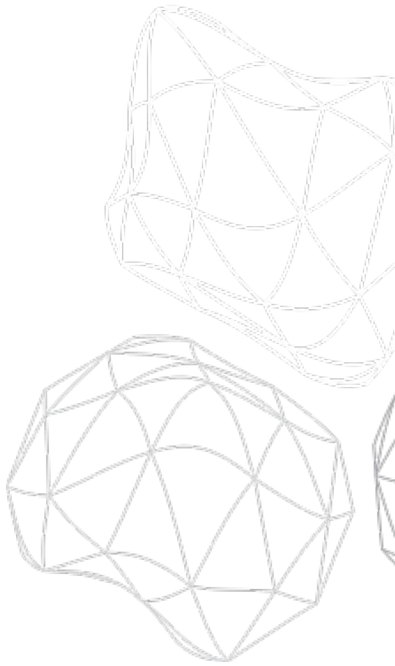
KSE · CS101

Sources: blocks/ in repo

Questions welcome



Appendix



Cross-entropy loss: derivation

Sigmoid squashes the raw score z into a probability p between 0 and 1

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Each pixel is a Bernoulli coin (y = 1 corner, 0 not). Independent pixels: multiply

$$P(y \mid x, \theta) = \prod_{i=1}^{N} p_i^{y_i} (1 - p_i)^{1 - y_i}$$

Take the log: the product becomes a sum

$$\ln P = \sum_i \left[y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \right]$$

Flip the sign and average: this is the loss

$$L_{BCE} = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \right]$$

Gradient for z: the sigmoid slope p(1–p) cancels the fractions

$$\frac{\partial \ell}{\partial z} = \left(-\frac{y}{p} + \frac{1 - y}{1 - p} \right) p(1 - p) = p - y$$

Intuition: it punishes surprise. A network that is sure about the wrong answer gets a huge loss; one that is right gets almost none.

Example: the pixel really is a corner (y = 1). The network says 69% and pays a small loss of 0.37. It says only 10% and pays 2.30, six times more.

Sources: research notes (blocks/03_tataryntsev.md), Bernoulli likelihood and maximum likelihood

Dice loss: derivation

Overlap of A (predicted corners) and B (true corners)

$$\mathrm{Dice}(A,B) = \frac{2\,|A \cap B|}{|A|+|B|}$$

Count with sums, so it can be differentiated

$$|A \cap B| \to \sum_i p_i y_i, \quad |A| \to \sum_i p_i, \quad |B| \to \sum_i y_i$$

High score should mean small loss

$$L_{Dice} = 1 - \frac{2\sum_i p_i y_i}{\sum_i p_i + \sum_i y_i}$$

In training it is added to cross-entropy, so both are minimized together

$$L = L_{BCE} + L_{Dice}$$

A = the pixels the network calls corners. B = the pixels that really are corners. |A| = how many pixels, A ∩ B = the ones in both.

Intuition: it asks how well two shapes overlap. Predicting no corners, or all pixels as corners, both score badly.

Example: p = 0.002 on 10,000 pixels with 20 corners. BCE is 0.014 (looks great), Dice loss is 0.998 (exposes the failure).

Sources: research notes (blocks/03_tataryntsev.md); Dice coefficient for sets

Return: derivation

Return G: all future rewards added up, nearer ones count more

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots$$

Recursion: reward now plus the discounted return from the next step

$$G_t = r_t + \gamma G_{t+1}$$

Goal: choose the policy θ with the best average return

$$J(\theta) = \mathbb{E}\left[G \right]$$

G = return (total future score) · r = reward at one step · t = step number · γ = discount between 0 and 1 (0.9: each later step counts 10% less) · E = average over many flights

Intuition: a reward now beats the same reward later, like money today versus a promise. γ sets how fast the future fades.

Example: rewards 1, 1, 1 and γ = 0.9. Last step G_2 = 1. Before it G_1 = 1 + 0.9·1 = 1.9. First step G_0 = 1 + 0.9·1.9 = 2.71.

Sources: research notes (blocks/03_tataryntsev.md); Nature, 2023

Imitation: compounding error

First mistake at step t, probability at most ε

$$P(\text{first mistake at } t) \le \epsilon$$

Worst case: no recovery, T−t+1 bad steps

$$\text{extra cost} \le \sum_{t=1}^{T} \epsilon\,(T-t+1)$$

Add up T + (T−1) + … + 1

$$= \epsilon\,\frac{T(T+1)}{2} \le T^2\epsilon$$

Independent mistakes would cost only

$$T\epsilon$$

Intuition: a driver trained only on perfect driving panics after the first slip, because nobody showed a recovery.

Example: T = 100, ε = 0.01: independent mistakes mean about 1 bad step, worst-case compounding up to 50 (bound 100). DAgger removes the mismatch.

Sources: Ross, Gordon, Bagnell, arXiv 1011.0686; research notes (blocks/03_tataryntsev.md)